Ders 4 Genişletilmiş Notu: Test Süreci Adımları, Saydam ve Kara Kutu Testleri

Ders 4 Genişletilmiş Notu: Test Süreci Adımları, Saydam ve Kara Kutu Testleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Proje Çalışmasının Çerçevesi
- 2. Yazılım İncelemesinin Tekrarı
- 3. Test ve V Model
- 4. Birim Testi ve Test Ortamı
- 5. Ara Birim Testi
- 6. Bütünlük (Bütünleme) Testi
- 7. Regresyon Testi
- 8. Sistem Testi
- 9. Onaylama Testi, Alfa ve Beta Testi, Kabul Muayenesi
- 10. Test Süreci Adımları ile Test Tipleri Arasındaki Fark
- 11. Saydam Kutu Testi
- 12. Performans Testinin Ayrıntısı
- 13. Kara Kutu Testi
- 14. Kara Kutu Yöntemleri: Eş Değerli Bölümleme, Sınır Değer Analizi, Keşif Testi
- 15. Sürekli Güncellenen Yazılımlarda Test
- 16. Temel Yol Testi ile Birim Testi Karışıklığı
- 17. Hata Giderme (Debugging)
- 18. Hata Düzeyleri
- 19. Dersin Sonundaki Tekrar: V Model, Doğrulama ve Geçerleme
- Örnek Sorular

1. Proje Çalışmasının Çerçevesi

Hoca derse, dönem projesinin genel çerçevesini hatırlatarak başladı. Projede her grup test edebileceği bir yazılım seçecek. Yazılımın alanı ve büyüklüğü serbesttir. Önemli olan, test aşamasında grubu sıkıntıya sokmayacak bir yazılım seçmektir. Hoca, seçilen yazılımın kodunun olmasını istedi, çünkü projede birim testi (unit test) yapılacak. Ayrıca bir sistem testi (system test) ve bir kullanıcı testi (user test) de istenecek.

Projeden beklenen çalışmalar şöyle sıralandı:

- **Gereksinim belgesi:** Bunun için derste ayrı bir bölüm yok. Sistem Analizi dersinde SRS belgesi adıyla, Yazılım Mühendisliği dersinde de daha ayrıntılı olarak görülmüştü. Hoca önce o notlara bakılmasını istedi; sorun olursa SRS belgesi üzerinden kısaca geçilebileceğini söyledi.
- **Birim testi:** Kod üzerinde yapılacak.
- **Sistem testi:** Herhangi bir araç (tool) kullanılabilir. Aracı grup kendisi araştırıp bulacak.
- **Kullanıcı testi:** Gereksinim belgesindeki fonksiyonel özelliklere bakılarak yapılacak.

- **Raporlama:** Kullanılan aracın tanımı, test verisi (test data), test senaryoları (test case). Kullanıcı testinde hangi test senaryosunun, birim ve sistem testinde hangi test verisinin kullanıldığı belirtilecek.
- **Test sonuç raporu.**
- **Test planı:** En başta yer alır. Nasıl bir tarih takvimi kurulduğu, ekipte kimin hangi işi yaptığı yazılır. Proje planı nasıl hazırlanıyorsa, test planında da testte yapılan işlerin nasıl yapıldığı anlatılır.

Hoca bazı yıllarda format ve örnek verdiğini, ama öğrencilerin araştırarak bulmasının daha iyi olacağını söyledi. Ayrıntılı kıstaslar sonraki haftalarda yazılı olarak verilecek.

2. Yazılım İncelemesinin Tekrarı

Hoca, yeni konuya geçmeden önce geçen haftanın konusu olan yazılım incelemesini (software inspection) tekrar etti. Test yöntemiyle birlikte yazılım kalitesini sağlamak için en çok kullanılan yöntem inceleme yöntemidir.

2.1 Statik ve Dinamik Geçerleme

Yazılım incelemesi şu belgeler üzerinde yapılabilir: gereksinim spesifikasyonu, yüksek seviye tasarım ve bununla ilgili belgeler, formal spesifikasyon belgeleri, ayrıntılı tasarım ve program. Program testi ise yalnızca gerçekten çalıştırılabilen programlarda ve prototip programlarda kullanılabilir. Bu nedenle:

- **Statik geçerleme ve doğrulama** incelemeyi kasteder.
- **Dinamik geçerleme ve doğrulama** testi kasteder.

2.2 İnceleme Süreci ve Fagan İncelemesi

Genel inceleme süreci üç temel adımdan oluşur: planlama ve hazırlık, inceleme ve toplantı, düzeltme ve tamamlama. Çok eskiden tanımlanmış bir inceleme türü olan Fagan incelemesi, bu basit adımları altı kademeye ayırarak daha ayrıntılı gösterir:

- Planlama: Neyin inceleneceği ve işin nasıl bölüşüleceği planlanır.
- Genel gözden geçirme: Ardından kişilere hangi kodu kimin inceleyeceği görevi verilir.
- Kişisel kod inceleme: Herkes kendisine düşen parçayı tek başına inceler.
- İnceleme toplantısı: Kod yazanlar, inceleyenler, kayıt tutan kişi ve kalite grubundan biri katılır.
- Düzeltme: Toplantı sonuçlarına göre yapılır.
- Tamamlama ve raporlama.

2.3 Kod Okuma

Kod incelemesinde en çok yapılan iş, kodu çalıştırmadan hem kurallar hem algoritma yapısı açısından değerlendirmektir. Hocaya göre etkili bir kod okuma tekniği, yukarıdan aşağı ve aşağıdan yukarı incelemenin birleşimidir. Program mümkün olduğunca küçük parçalara, yani alt programlara, fonksiyonlara ve prosedürlere ayrılır. Önce bu parçaların kendi içindeki bütünlüğü ve doğruluğu incelenir. Sonra parçaların programın bütünüyle uyumuna bakılır.

İnceleme ekibinde inceleme sorumlusu, inceleyiciler, kod yazanlar, kod okuyucu ve kayıtçı bulunur. Sonunda bir teknik inceleme özet raporu hazırlanır. Raporda neyin gözden geçirildiği, kimin gözden geçirdiği, hangi hata ve eksiklerin bulunduğu ve neler yapılması gerektiği yazılır. Rapor üç sonuçtan biriyle biter:

- Kod değişiklik yapılmadan kabul edilir.
- Belirtilen önemsiz hataların belli bir sürede düzeltilmesi koşuluyla kabul edilir.
- Reddedilir; düzeltmeden sonra yeniden inceleme yapılır.

3. Test ve V Model

Test, bir programdaki hataları bulmak amacıyla yapılan işlemlerdir. Testte fonksiyonel, performans, dayanıklılık ve yapısal bakımdan yeterlilik denetlenir.

3.1 V Modelin Yapısı

Hoca geçen hafta görülen V yaklaşımını (V model) yeniden ele aldı. Model hem doğrulama ve geçerleme (verification, validation) adlarının baş harfi yüzünden hem de şekli yüzünden V adını alır. V'nin sol kolunda yazılım geliştirme etkinlikleri, sağ kolunda test süreci adımları bulunur.

Sol kol yukarıdan aşağı şöyledir:

- Müşteri gereksinimleri alınır.
- Ürün spesifikasyonu belirlenir.
- Yüksek düzey tasarım, yani genel tasarım yapılır.
- Alçak düzey tasarımda her programın algoritması tasarlanır.
- Kodlama yapılır.

Kodlamanın hemen ardından kodu yazanlar birim testini yapar ve test süreci başlar. Sağ kol aşağıdan yukarı birim testi, gerekirse bileşen testi (bileşen model yapısı varsa), bütünlük testi, sistem testi ve işlemsel kullanım için kullanıcı onay (kabul) testi olarak ilerler.

Hoca, bu sıranın şelale (waterfall) modeline göre olduğunu belirtti. Günümüzde daha çok kullanılan çevik (agile) yöntemlerde sağ taraftaki test adımları sol taraftaki geliştirme adımlarıyla birlikte yürütülür. Kodun bitmesi beklenmez.

3.2 Ayrıntılı V Model: Regresyon ve Kabul Testi

V model biraz daha ayrıntılandırılınca bütünlük testinin hemen üstüne regresyon testi (regression testing) gelir. Bütünlük testi tamamlandıktan sonra alttaki birimlerde değişiklik olmuşsa bütünlük testinin yeniden tamamlanması gerekir; buna regresyon testi denir. Bütünlük testi ve onu tamamlayan regresyon testinden sonra yazılım doğru çalışır. Fakat yazılımın işletim sistemiyle ve veritabanı sistemiyle ne kadar uyumlu çalıştığına bakmak için sistem testi yapılır. Sistem testinin üstüne de kabul testi (acceptance test) gelir. Burada kullanıcıdan bir onay beklenir ve bu onay alfa ve beta testi biçiminde alınır.

Hoca slaytta regresyon ve kabul testini bütünlük ve sistem testinin "yanına" yazdığını, doğrusunun "üstüne" olduğunu söyledi.

4. Birim Testi ve Test Ortamı

Hoca geçen hafta görülen birim testini de kısaca tekrar etti.

Birim testi, yazılım tasarımının en küçük birimi olan modül üzerinde uygulanır. Ayrıntılı tasarım tanımlarına dayanır. Algoritmanın içinde hata olup olmadığına bakılır ve önemli kontrol yolları test edilir. Birim testinde saydam kutu testi uygulanır. Test çok sayıda modül üzerinde paralel olarak yürütülebilir.

> **Hocanın vurgusu:** Birim testini test grubu değil, kodu geliştiren kişi yapar.

Birim testinde şunlar test edilir:

- Modülün ara birimi (interface).
- Yerel (lokal) veri yapıları.
- Kontrol yapıları arasındaki ana yollar.
- Modülün sınırları ve veri sınırları.

4.1 Test Sürücü ve Kukla Modül

Birim testinin ortamında iki yapay program bulunur:

- **Test sürücü (test driver):** Test edilen birimi çağıran programdır. Birime test verisini gönderir ve sonuçları toplar.
- **Kukla modül (stub):** Test edilen birimin çağırdığı modüller henüz test edilmemişse onların yerine konan yapay modüldür.

Dersin sonundaki tekrarda hoca bunu şöyle özetledi: Birim testi yapılırken birimi çağıran program test sürücüdür, birimin çağırdığı program kukladır. İkisi de yapay, kukla (dummy) alt programlardır.

4.2 Test Senaryosu

Test senaryosu (test case), belirli bir program yolunu işletmek ya da özel bir gereksinime uygunluğu onaylamak amacıyla düzenlenen bir dizi test verisidir. Bu test verisi ve onunla yapılan işlemlerin hepsine test senaryosu denir. Hoca, projede de seçilen yazılım için hangi test verisinin kullanıldığının ve buna nasıl karar verildiğinin yazılmasını istedi.

Test programları da diğer yazılımlar gibi planlanır: Bir test planı hazırlanır ve yazılım tasarımıyla birlikte değerlendirilir. Modüller genellikle birbirinden bağımsız olmadığı için test sürücüleri ve kuklalar yazılımın içinde geliştirilir.

5. Ara Birim Testi

Bütünlük testinden önce ara birim testi (interface test) yapılabilir. Birim testinde modülün içi değerlendirilmiş ve modül çalıştırılmıştır. Şimdi modülün hangi modüllerle ara birimi olduğuna bakılır. Örneğin A, B ve C modüllerinin birbirine veri göndermesi ve aralarındaki kontrol yapıları, bu ilişkileri değerlendirecek test verileriyle, yani test senaryolarıyla denenir. Ara birim testi bütünlük testine bir hazırlıktır.

6. Bütünlük (Bütünleme) Testi

Modüller birbirinden bağımsız değildir; birbirine yazılım ara birimleriyle bağlıdır. Bu nedenle tek başına doğru çalışan modüller birleştirildiğinde hatalı çalışabilir. Hoca birleştirme sırasında çıkabilecek sorunları şöyle saydı:

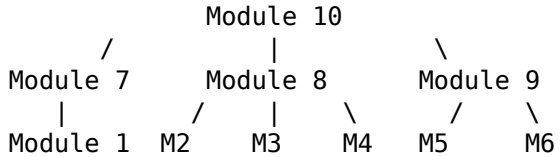
- Modüller arasında veri kaybı olabilir.
- Farklı modüller birbirini ters etkileyebilir.
- Alt fonksiyonlar birleştirildiğinde beklenen ana fonksiyon gerçekleşmeyebilir.
- Her modülde kabul edilen bir hata toleransı vardır. Bu toleranslar birleşince eklenerek büyüyebilir.
- Genel (ortak) veri yapıları sorun yaratabilir.

  Hocanın vurgusu: Tek başına doğru çalışan modüller birleştiğinde hatalı çalışabilir. Bütünlük testinin nedeni bu hataları bulup yok etmektir.

6.1 Bütün Olarak ve Artırmalı Sınama

Bütünlük testi iki ayrı biçimde yapılabilir: bütün olarak ya da artırmalı (incremental) sınama ile. Bütün olarak sınamak daha zordur. Artırmalı sınamada modüller teker teker bağlanır. Bu bağlama yukarıdan aşağı ya da aşağıdan yukarı yapılabilir.

Hocanın slayttaki örneği şu modül ağacıydı:



6.2 Yukarıdan Aşağı Bütünleme

Yukarıdan aşağı bütünlemede (top-down integration) ana modülü çalıştırabilmek için önce Module 10, Module 7, 8 ve 9 ile birlikte çalıştırılır. Sonra Module 7, Module 1'i de çalıştıracak biçimde bir araya getirilir. Module 8'i çalıştırmak için Module 2, 3 ve 4 bir araya getirilir. Böylece test 1, test 2, test 3, test 4 diye ilerlenerek üstteki bütünün altındaki yapılar sırayla eklenir. Henüz eklenmemiş alt modüllerin yerine kukla modül kullanılır.

6.3 Aşağıdan Yukarı Bütünleme

Aşağıdan yukarı bütünlemede (bottom-up integration) önce en alttaki modüller test edilir. Önce en soldaki modülün altındaki modül (Module 1) test edilir ve Module 7 yerine konur. İkinci blok olarak Module 2, 3 ve 4 test edilip birleştirilir ve bunları çağıran Module 8 de test edilmiş olur. Üçüncü işlem aynı biçimde yapılır. En sonda bir üst seviyedeki modüller birleştirilerek test edilir. Hoca dersin sonundaki tekrarda bu birleştirmenin kümeler (cluster) aracılığıyla yapıldığını ve program yukarıya doğru tamamlandığını söyledi. Alt modülleri çağırmak için test sürücü kullanılır.

6.4 Sandviç Testi

Üçüncü bir yol da sandviç testidir: Hem yukarıdan aşağı hem aşağıdan yukarı bütünleme birlikte uygulanır.

7. Regresyon Testi

Regresyon testinde, test edilmiş bir programda değişiklik veya ekleme yapılmışsa programın tümü bir kez daha test edilir. Uygulama ortamında gerekli değişiklikler ve sabitlemeler yapıldıktan sonra da regresyon testi kapsamında yeniden test yapılır. Kısacası regresyon testi, önceden test edilmiş bir yazılımın çeşitli değişikliklerden geçtikten sonra hatasız çalıştığını sağlamak için yeniden test edilmesidir.

Derste bu konu bir soru olarak da soruldu. Bir öğrenci regresyon testini, bir uygulamanın hataları düzeltildikten sonra verilen yeni sürümünde o hataların gerçekten düzeltilip düzeltilmediğinin kontrolü olarak tanımladı. Hoca bu tanımı kabul etti ve ekledi: Geliştirme sürecinde altta bir hata olmuşsa bütünlük testi tekrar edilir; bu da regresyon testidir.

8. Sistem Testi

Bilgisayar sistemi donanım ve yazılım sistemlerinden oluşur. Yazılım alt sisteminin kendi başına test edilmesi yeterli değildir; bilgi sistemi içinde denetlenmesi gerekir. Sistem testinin amacı, sistemin bütün öğelerinin uygun biçimde bir araya getirildiğini ve her birinin işlevini tam olarak yerine getirdiğini test etmektir.

Sistem testi türleri şunlardır:

- **Düzeltme testi:** Hoca bu testi yalnızca adıyla saydı.
- **Güvenlik testi (security testing):** Sistemin dış müdahalelerden ne kadar korunabildiğini kanıtlar. Hoca tekrarda bunu zararlı dış müdahalelere ve bilgi hırsızlığına karşı korunma olarak açıkladı.
- **Dayanıklılık testi (stress testing):** Sistemin miktar, frekans ya da hacim bakımından anormal biçimde yüklenmesi durumundaki dayanıklılığını ölçer. Örneğin fazla veri girişi olması ya da çok sayıda kullanıcının aynı anda yüklenmesi.
- **Yetenek (performans) testi (performance testing):** Gerçek zamanlı ve gömülü sistemlerde yazılımın işlem süresinin bilgisayara dayalı sisteme uygunluğunu test eder. Örneğin cevap süresi gibi durumlarda uygulanır.

  Ek bilgi (derste anlatılmadı): “Düzeltme testi” büyük olasılıkla İngilizce kaynaklardaki recovery testing (kurtarma testi) karşılığıdır. Bu testte sistem bilerek arızaya uğratılır ve ne kadar sürede, ne kadar doğru biçimde toparlandığı ölçülür.

Performans testi her test basamağında uygulanabilir. Örneğin birim testi yapılırken modülün performansı da sistem testine bırakılmadan ölçülebilir. Bütünlük ve regresyon testinde de test araçları kullanılır.

> **Hocanın vurgusu:** Sistem testinde mutlaka bir araç (tool) kullanmak gerekir. Çünkü çok sayıda kişinin bir yazılımı aynı anda kullanmasını sağlamak çok güçtür; bu ancak destek yazılımıyla yapılabilir.

9. Onaylama Testi, Alfa ve Beta Testi, Kabul Muayenesi

9.1 Onaylama Testi ve Konfigürasyon İncelemesi

Bütünleme testinin sonunda yazılım bir paket haline gelmiştir. Ara birim hataları bulunmuş ve düzeltilmiş, yazılımın sistem üzerinde çalışması değerlendirilmiştir. Bundan sonra onaylama testi (validation test) yapılır. Onaylama testi, yazılımın müşteri ve kullanıcı beklentilerini karşılayıp karşılamadığını anlamaya yöneliktir.

Onaylama testinde bütünleşmiş yazılım, gereksinimler ve kullanıcı belgeleri birlikte kullanılır. Sorulan sorular şunlardır: "İstediğim gereksinimlerin hepsi bu bütünleşmiş yazılımda var mı? Kullanıcı belgeleri beni doğru yönlendiriyor mu?"

Bunun yanında konfigürasyon (düzen) incelemesi yapılır. Kullanıcı belgeleri, tasarım belgeleri, kayıtlar ve test belgeleri birleştiğinde birbiriyle uyumlu ve teslim edilebilir durumda mı, buna bakılır. Hocanın tekrardaki ifadesiyle, yazılımcıların belirtilen bütün elemanları teslim edip etmediği kontrol edilir.

Onaylama testi ve konfigürasyon incelemesi birlikte kabul muayenesini oluşturur. Kullanıcı kabul testinde müşteri tarafı yazılımı bütün alt programlarıyla, gereksinimleri ve kullanıcı belgelerini inceleyerek onaylar. Teknik ekip de konfigürasyonu inceler: Kullanıcı belgeleri, tasarım belgeleri, çalışma kayıtları ve test belgeleri var mı? İki incelemenin sonucu da uyumlu çıkarsa uygulama kabul muayenesini geçmiş olur ve yazılım teslim edilebilir.

9.2 Alfa Testi

Sistem testinden sonra, onaylama testini sağlayabilmek için arada bir alfa testi yapılır. Alfa testi bir kullanıcı testidir, ama kullanıcıya benzer bir kullanıcı senaryosu kurulur. Kullanıcıya benzer bir grup, yazılım evinin içinde gereksinim belgelerini takip ederek ya da kullanıcıya verilecek yardım dosyalarını ve yardım menülerini kullanarak beklenen işlemlerin gerçekleşip gerçekleşmediğine bakar.

Bu işi yazılım ekibinde yazılımı geliştiren kişiler değil, kullanıcıya benzer bir grup yapar. Bu grupta özellikle dikkat ve takip yeteneği olması gerekir. Alfa testinden sonra yazılım kullanıcıya teslim edilir.

9.3 Beta Testi

Beta testinde yazılım kullanıcıya benzer bir gruba değil, potansiyel kullanıcılara verilir ve onlar yazılımı kullanarak değerlendirir. Hoca, sistem yazılımlarının ya da günlük hayatta kullanılan yazılımların beta sürümlerinin herkesin karşısına çıktığını hatırlattı. Bir beta sürümü belli bir süre kullanılıp hata bildirildiğinde, bu hata uygulama için gerçekten önemli ve düzeltilmesi gereken bir hataysa bildirene ödül de verilebiliyor. Alfa ve beta testi böylece onaylama aşamasına yardımcı olur ve kullanıcıdan gelen geri dönüşleri iyileştirir.

10. Test Süreci Adımları ile Test Tipleri Arasındaki Fark

Aradan sonra hoca şimdiye kadar görülenleri bağladı. Test süreci adımları, test işinin hangi sırayla yapılacağını anlatır: birim testi, bütünlük testi, sistem testi ve kullanıcı testi. Fakat bu adımlarda hangi tip testlerin yapıldığı ayrı bir sorudur. Bu soru test tiplerine götürür.

- **Kara kutu testi (black box testing):** Fonksiyonel, performans ve dayanıklılık testleri sistemin dış spesifikasyonlarına ve gereksinimlerine dayandırıldığı için bu ad verilir.
- **Saydam kutu testi (white box / glass box testing):** Yapısal denetimde, modül düzeyinde programın deyimleri, dalları ve akış yolları test edilerek iç yapısı incelenir.
- **Gri kutu testi (gray box testing):** İkisinin karmasıdır. Test verisi saydam kutu biçiminde hazırlanıp test kara kutu biçiminde yapılabilir ya da tersi.

  Hocanın vurgusu: Test tipi ile test süreci adımı farklı şeylerdir. "Hangi sırayla test ederiz?" sorusunun cevabı test süreci adımlarıdır. "Bu adımların her birinde hangi testleri uygularız?" sorusunun cevabı test tipleridir: kara kutu, saydam kutu ve gri kutu.

11. Saydam Kutu Testi

Saydam kutu testinde işlemsel tasarımın kontrol yapısı kullanılır. Bu test şunları sağlar:

- Bir modüldeki bütün bağımsız yollar en az bir kez çalıştırılır.
- Bütün mantıksal kararların doğru ve yanlış durumları denenir.
- Bütün döngülerin kendi içinde ve sınırlarında işlerliği sağlanır.
- İç veri yapıları denenerek geçerlilikleri güvence altına alınır.

Bunları yapmak için farklı saydam kutu test tipleri vardır. Derste ikisi örneklendi: temel yol testi ve döngü testi.

11.1 Temel Yol Testi

Temel yol testi (basis path testing), işlemsel tasarımın mantıksal karmaşıklığını ölçmeye ve bu ölçüye göre uygulama yolları için bir temel grup oluşturmaya dayanır. Programda kaç tane akış yolu olduğu döngüsel karmaşıklık (cyclomatic complexity) değeriyle bulunur. Test programları, test sırasında programdaki her deyimi en az bir kez çalıştırarak dener.

Temel yol testinin adımları şöyledir:

- Ayrıntılı tasarıma ya da kaynak programa dayanarak bir akış grafı çizilir.
- Akış grafı üzerinde McCabe döngüsel karmaşıklık ölçüsü bulunur.
- Bağımsız yolların temel grubu ve düğümleri belirlenir.
- Temel gruptaki her yolu denemek için birer test programı düzenlenir. Bunun içinde her yolu geçecek biçimde test verisi hazırlamak da vardır.
- Test programı uygulanır ve sonuç beklenen sonuçla karşılaştırılır.

Hoca hedefi şöyle özetledi: Programda kaç akış yolu olduğunu McCabe değeriyle hesaplamak, her yol için uygun test verisini oluşturmak ve programı bu verilerle temel yol sayısı kadar çalıştırmak. Böylece her yolun testi yapılmış olur.

McCabe ölçüsünün nasıl hesaplanacağı yazılım ölçümü konusunda anlatılacak. Hoca bunu iki kez anlatmamak için bu bölümde hesap vermedi.

> **Ek bilgi (derste anlatılmadı):** Döngüsel karmaşıklık akış grafı üzerinden $V(G) = E - N + 2$ formülüyle hesaplanır. Burada E kenar sayısı, N düğüm sayısıdır. Tek giriş ve tek çıkışlı bir programda bu değer, karar düğümü sayısı P ile $V(G) = P + 1$ olarak da bulunabilir. Elde edilen değer, temel gruptaki bağımsız yol sayısını verir.

11.2 Döngü Testi

Döngü testi (loop testing) de bir saydam kutu testidir ve temel yol analizine ek olarak yürütülür. Döngü içinde bir giriş ve bir çıkışlı olarak soyutlanan bütün yollar birer döngü olarak ayrıca sınanır. Amaç, döngü içindeki başlama hatalarını, indis hatalarını, artırma hatalarını ve döngü sınırı hatalarını bulmaktır. Test sonunda döngü yapısının geçerliliği onaylanmış olur.

Programdaki bütün döngülerin test edilmesi gerekir. Hoca slayttaki döngü türlerini şöyle açıkladı:

- **Basit döngü:** Başlama hatası, devam hatası, sonsuz döngü hatası var mı? Başlangıç, bitiş ve indis değerlerinde ya da bir while döngüsünün koşulunda döngüyü sonsuza götürecek bir hata var mı?
- **Yuvalanmış (iç içe) döngü:** İç içe döngülerin birbiriyle ilişkisinde ve indisleri arasında hata var mı?
- **Birbirine bağlı (ardışık) döngüler:** Devam konusunda hata var mı? Bir döngüden alttaki döngüye yanlış atlayan bir yapı var mı?
- **Yapısal olmayan döngüler:** Bunlar da incelenir ve değerlendirilir.

Hoca saydam kutu testini toparlarken, bunun bir iç yapı testi olduğunu söyledi. İç yapı, kontrol deyimleri, kontrol akışı ve döngülerden oluşur. Bu nedenle hem kontrol akışına bakan temel yol testinin hem de döngü testinin yapılması gerekir.

12. Performans Testinin Ayrıntısı

Kara kutu testine geçmeden önce hoca, sistem testinde en çok yapılan iş olan performans testinin ayrıntısını verdi. Performans testinde yapılanlar şunlardır:

- Performans seviyelerinin belirlenmesi.
- Belirli bir yük altında sistemin çalışma zamanı performansının ölçülmesi.
- Sistem ayarlamaları (tuning).

Performans testi şu sorulara cevap verir:

- Sistem performans gereksinimlerini karşılıyor mu?
- Normal şartlar altında sistem nasıl davranıyor?
- Sistem trafiğindeki artışlar işlem süresini ve işlevselliği nasıl etkiliyor?
- Hangi kullanıcı seviyesinde performans sorunları başlıyor?
- Performanstaki düşüş sistemin hangi bileşeninden kaynaklanıyor?

13. Kara Kutu Testi

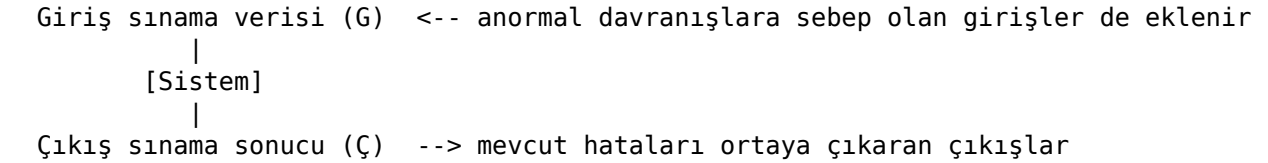
Kara kutu testi, test tipleri içinde bir dış testtir. Dış testten kasıt, yazılımın iç yapısının değil, dışarıya verdiği tepkilerin incelenmesidir. Kara kutu testi yazılımın bütünlenmesi sırasında uygulanır ve yazılım ara birimi üzerinde yapılır. Yani hem bütünleme testi hem sistem testi sırasında yapılır. Gösterilmek istenenler şunlardır:

- Yazılım işlevlerini yerine getiriyor mu?
- Girdileri kabul ediyor mu?
- Çıktıları doğru olarak üretiyor mu?

Kara kutu testinde yazılımın mantıksal iç yapısıyla ilgilenilmez; temel sistem yapısı denenir. Bu nedenle kara kutu ve saydam kutu testleri birlikte uygulanır. Ara birim testinde ve bütünlük testinde ikisi de yapılır.

13.1 Kara Kutu Test Modeli

Hoca kara kutu test modelini slayttaki şekil üzerinden anlattı:



Giriş test verisi oluşturulup sisteme yüklenir. Yalnızca doğru girişler değil, anormal davranışlara sebep olan girişler de özellikle verilir. Hatalı bir test verisi gönderildiğinde sistemin nasıl etkileneceğine bakılır: Sistem tamamen kesilecek mi, yoksa geliştirilen yazılım kesilmeden hata bilgisi mi verecek? Çıkış tarafında da mevcut hataları ortaya çıkaran çıktılar değerlendirilir.

> **Hocanın vurgusu:** Kara kutu testinde, saydam kutu testinden farklı olarak, sistemin iç yapısıyla hiç ilgilenilmez. Yalnızca gönderilen girdilerin hangi çıktılara yol açtığına bakılır.

13.2 Kara Kutu Testinin Kapsamı ve Bulduğu Hatalar

Kara kutu yöntemleri yazılımın işlevsel gereksinimlerini denemeye yöneliktir. İşlevsel olmayan gereksinimler de sistem testi kapsamında yine kara kutu testiyle denenebilir. Programın bütün işlevsel gereksinimlerini denemek için bir dizi girdi durumu, yani test senaryoları oluşturulur.

> **Hocanın vurgusu:** Kara kutu testi saydam kutu testinin alternatifi değildir. İkisi birbirini tamamlar; ikisi birlikte kullanılınca farklı karakterde hatalar bulunur.

Kara kutu testiyle bulunan hatalar şunlardır:

- Hatalı ve eksik işlevler: Girdi verildiğinde istenen çıktı alınamaz.
- Ara birim hataları.

- Veri yapılarında ve veritabanı erişiminde hatalar.
- Performans hataları.
- Başlama ve bitirme hataları.

Başlıca kara kutu test yöntemleri eş değerli bölümleme, sınır değer analizi, neden-sonuç grafiği çizimi (cause-effect graphing) ve veri onaylama testidir. Derste yalnızca ilk ikisi üzerinde duruldu.

14. Kara Kutu Yöntemleri: Eş Değerli Bölümleme, Sınır Değer Analizi, Keşif Testi

14.1 Eş Değerli Bölümleme

Eş değerli bölümleme (equivalence partitioning) yönteminde programın girdi alanı, test programı oluşturulabilecek veri sınıflarına bölünür. Böylece her test programı belirli bir sınıftaki hataları ortaya çıkarır ve daha az sayıda test programıyla yetinilir.

Hedef, test verisini rastgele seçmemektir. Test edilecek sistemin girdileri veri sınıflarına bölünür ve her sınıftan veri gönderilir. Böylece veri hep aynı bölgeden seçilmez; farklı bölgelerden girilen verilerle sonuçlara bakılır. Slayttaki şekilde geçerli girdiler ve geçersiz girdiler ayrı gösteriliyordu. Kara kutu test modelinde söylendiği gibi geçersiz girdiler de özellikle gönderilir. Geçerli girdiler ise “şu aralıktaki veri, şu aralıktaki veri” diye aralıklara bölünür. Sonra sisteme verilen girdilere hangi çıktıların karşılık geldiği değerlendirilir.

Yaş Örneği Hoca yöntemi şu örnekle açıkladı:

- **Verilen:** Programa girdi olarak verilen yaş, 0 ile 99 arasında bir değerdir. Dört test yapılacaktır.
- **Rastgele seçim:** Dört değer rastgele seçilirse örneğin 1, 97, 87 ve 99 gelebilir.
- **Bölümleme:** Yaş değeri gruplara bölünür: 0-15, 15-45, 45-80, 80-99. Her gruptan bir değer seçilir.
- **Sonuç:** Hocaya göre ikinci yolla daha iyi bir test yapılmış olur.

Rastgele seçilen dört değeri hocanın sınıflarına yerleştirince farkın nedeni görülür (ara adım eklendi):

Değer	Düştüğü sınıf
1	0-15
97	80-99
87	80-99
99	80-99

Dört testten üçü aynı sınıfa düşer; 15-45 ve 45-80 sınıfları hiç denenmez. Bölümleme yapıldığında ise dört testin her biri farklı bir sınıftan gelir.

> **Ders sonrası not:** Hoca sınıfları 0-15, 15-45, 45-80, 80-99 diye söyledi. Bu aralıklarda 15, 45 ve 80 değerleri iki sınıfa birden düşer; eş değerli bölümlemede sınıflar çakışmamalıdır. Çakışmasız bir bölümleme 0-14, 15-44, 45-79, 80-99 olur. Ayrıca negatif değerler ve 99’dan büyük değerler geçersiz sınıf olarak ayrıca denenmelidir.

Hoca bunun çok genel bir örnek olduğunu, daha kritik yazılımlarda ya da sayısal işlemlerde sınıfların farklı belirlenebileceğini söyledi. Hedef, girdi uzayındaki değerleri sınıflara bölüp her sınıftan test verisi seçmektir.

Optimum Test Sayısı Her test bir maliyet getirir. Ne kadar çok test yapılırsa o kadar iyidir, ama bunun da bir bedeli vardır. Hoca, test maliyetini en iyi noktaya çekecek optimum test sayısını yapay zekâ yöntemleriyle hesaplamanın bir araştırma konusu olduğunu söyledi. Yapılacak iş, bu optimum test sayısını aşmayacak biçimde sınıf sayısını belirlemek ve her sınıftan en az bir test verisiyle deneme yapmaktır.

Dersin sonunda bir öğrencinin bu yöntemle ilgili sorusuna hoca şöyle cevap verdi:

Soru: Eş değerli bölümlemede sınıf oluşturmanın amacı nedir? **Cevap:** Amaç gereksiz test sayısını azaltmaktır. Aralıklar oluşturularak gereksiz testler azaltılır ve test daha verimli hale gelir. Bu yüzden sınıflar oluşturulur.

14.2 Sınır Değer Analizi

Sınır değer analizi (boundary value analysis) da bir kara kutu yöntemidir. Hoca bunu kabaca şöyle anlattı: Hata genellikle sınırlarda, en uç noktalarda olur. Bu yüzden önce sınırlara bakılır.

Sınır değer analizinde test verisi, girdi alanının ya da çıktı alanının sınıflarının, veri yapılarının ve alt program parametrelerinin sınırlarından veya uç değerlerinden seçilir. Genellikle en büyük, en küçük ya da en az beklenen değerler ve bunlarla ilgili parametreler test verisi olarak seçilir.

Hataların merkezden çok kenarlarda toplandığına dair bir kanı vardır. Bu nedenle sınır değerlerini denemek için bazı test programları ve araçlar geliştirilmiştir. Bu programlar girdi alanında olduğu kadar çıktı alanında da uygulanır. Kısacası "test verisini nasıl seçeceğiz?" sorusuna bu yöntemin cevabı, sınır değerlerini seçmenin daha uygun olduğudur.

> **Ek bilgi (derste anlatılmadı):** Sınır değer analizi genellikle eş değerli bölümlemeyle birlikte kullanılır: Önce sınıflar belirlenir, sonra her sınıfın alt ve üst ucu ile bu uçların hemen dışındaki geçersiz değerler denenir. Yukarıdaki yaş örneğinin çakışmasız biçiminde bu yaklaşım 0, 14, 15, 44, 45, 79, 80, 99 değerlerini ve geçersiz uçlar olarak -1 ile 100'ü seçer.

14.3 Keşif Testi

Keşif testi (exploratory testing), ürün spesifikasyonu ya da gereksinim belgesi olmadan, yazılımın kendisini ürün spesifikasyonu gibi kullanarak yapılan testtir. Keşif testinde test edenin çok ayrıntılı bilgisi yoktur; yazılım genel olarak çalıştırılıp kullanılır. Hoca bunu ürün spesifikasyonu ya da gereksinim belgesi olmadan yapılan, yazılımın genel olarak çalıştırılıp incelendiği dinamik bir kara kutu testi olarak tanımladı.

15. Sürekli Güncellenen Yazılımlarda Test

İkinci aradan sonra bir öğrenci şu soruyu sordu:

Soru: Spotify gibi bir uygulama piyasaya sürülmeden önce derste öğrendiğimiz testlerden geçiyor. Peki sürekli kendini güncelleyen ve piyasadan hiç çekilmeyen bir uygulamada bu test aşamaları her güncellemeden sonra baştan mı yapılıyor, yoksa yalnızca bir kısmı mı?

Cevap: Testler her seferinde baştan yapılmaz. Burada yardımcı programlara çok ihtiyaç vardır. En son yapılan testler ve kullanılan test verileri saklanır. Sürüm yönetimi (release management) konusu da bu yüzden son yıllarda önem kazanmıştır: Yeni bir sürüm çıktığında bunun nasıl yönetileceği sorusu. Sürüm yönetimi, konfigürasyon (düzen) yönetimiyle birlikte yürür. Konfigürasyon yönetiminde uygulamada hangi anda hangi değişikliğin yapıldığı izlenir. Bir yazılımın elemanları yalnızca kod değildir; gereksinim belgesi, tasarım belgesi, test senaryoları belgesi ve test belgeleri de vardır ve hepsi birlikte gözden geçirilir. Yeni bir sürüm yapıldığında konfigürasyon yönetimiyle birlikte çalışılır. Şimdiye kadar yapılan bütün testler, uygulanan test verileri ve test senaryoları arşivde saklı olduğu için testlerin hepsi yeniden yapılmaz; eklemeler var olanın üzerine konarak devam edilir.

16. Temel Yol Testi ile Birim Testi Karışıklığı

Hoca, bir öğrencinin yazılı sorusu üzerine temel yol testi ile birim testinin karıştırılmış olabileceğini söyledi ve ikisini ayırdı:

- **Temel yol testi** bir saydam kutu test tipidir. Birim testinin içinde uygulanır.
- **Birim testi** bir test süreci adımıdır. Önce modül test edilir; modülün hem algoritmasının iç yapısı, hem döngüleri, hem veri yapıları kontrol edilir.

  Hocanın vurgusu: Birim testinde, saydam kutu test tipine giren temel yol testi uygulanır. Temel yol testi bir test tipi, birim testi bir test adımıdır; ikisini karıştırmayın.

17. Hata Giderme (Debugging)

17.1 Test ile Debug Farkı

Test, hatalı bölümleri bulma çalışmasıdır. Hata giderme (debugging) ise test sonucunda saptanan hata ve eksiklerin nedeninin bulunup düzeltilmesidir. Önce hatanın hangi belirtilerle görüldüğü anlaşılır, sonra hata düzeltilir.

17.2 Hata Kaynakları

Hatalar farklı nedenlerle ortaya çıkabilir. Slayttaki şekilde hata kaynakları gereksinim analizinden kaynaklanan hatalar, tasarım hataları ve kod hataları olarak gösteriliyordu. Gereksinim hataları, kullanıcının istekleri ile yazılımın gerçekleştirdiği işlemler arasındaki farklılıklardır. Bunun nedeni iki taraflıdır: Kullanıcı isteklerini doğru anlatamamış, yazılımcı da onu doğru yönlendirememiştir.

17.3 Çevik Yöntemlerde Gereksinim Hataları

Hoca, çevik (agile) modellerin yaygınlaşmasıyla gereksinim analizinden gelen hataların azalmaya başladığını söyledi ve nedenini sınıfa sordu. Öğrencilerden gelen cevaplar şunlardı:

- Gereksinim ile kod arasında geçen süre kısadır. Hoca bunun bir neden olabileceğini kabul etti.
- Gereksinim analizi kısmı daha kısadır.
- İteratif bir yapıda, küçük adımlarla ilerlenir. Hoca bunun özellikle XP (Extreme Programming) için geçerli olduğunu söyledi.

Bir cevapta spiral model de anıldı. Hoca spiral modelde her adımda kontrol, inceleme ve risk analizi olduğu için gereksinim hatalarının daha az olduğunu, ama sürecin çok zorlu ve çok maliyetli olduğunu belirtti. Asıl sorunun çevik yöntemlerin şelale modelinden farkı olduğunu hatırlattı.

Hocanın aradığı cevap sonunda geldi: Müşteri aktif olarak gereksinim kısmına katılır. Müşteri ürünü yalnızca bittiğinde değil, aşama aşama her halinde görür. Hoca bunun yalnızca ürünü görmek ya da kontrol ettirmek anlamına gelmediğini vurguladı.

> **Hocanın vurgusu:** Çevik yöntemlerde müşteri ya da müşteri temsilcisi ekibin aktif bir parçasıdır. Gereksinim analizindeki hataların azalmasının temel nedeni budur.

18. Hata Düzeyleri

Bir yazılım bittiğinde bir hata dökümü vardır; debug işlemi yapılmış, test sonucunda bulunan ve düzeltilecek hatalar ortaya çıkmıştır. Bu hatalar farklı biçimlerde adlandırılabilir. En kapsamlı sınıflama şu düzeylerden oluşur: ölümcül, kritik, büyük, orta, küçük ve görünüm (kozmetik) hataları. Yazılımın devamı hakkındaki kararlar bu düzeylere göre verilir. Yalnızca orta, küçük ve görünüm hataları çıkmışsa yola devam edilir. Kritik ya da ölümcül bir hata varsa işlemler durdurulur.

Düzey	Test	Teslim
Ölümcül	Testin devamını engeller; test durdurulur	Teslim edilmez
Kritik	Test devam eder; bazı kurumlarda test durdurulur	Düzeltilmeden kesinlikle teslim edilmez
Büyük	Test devam eder	Hata giderilmeden teslim zararlı olur; mümkünse teslim edilmez
Orta	Test devam eder	Ürün bu hatayla teslim edilirse sorun çıkabilir
Küçük	Test devam eder	Teslim edilebilir; teslimden sonra düzeltilmeye çalışılır

Düzey	Test	Teslim
Görünüm (kozmetik)	Test devam eder	Teslim edilebilir; renk, font, büyüklük gibi kullanıcının farklı görmek isteyeceği hatalardır

Hoca kritik hatayı şöyle açıkladı: Kritik bir hata varsa test devam ettirilir, ama hatanın düzeltilmesi için çalışılır ve hata düzeltilmeden yazılım kesinlikle teslim edilmez.

19. Dersin Sonundaki Tekrar: V Model, Doğrulama ve Geçerleme

Hoca dersin sonunda karışıklığı önlemek için test süreci adımlarını V model üzerinden yeniden geçti.

19.1 V Modelin Gösterdiği Üç Şey

V model yalnızca sağ taraftaki test süreci adımlarını göstermez. Hocaya göre şekil üç şeyi birlikte gösterir:

- **Test süreci adımlarının sırası:** Birim testi, bütünlük testi, sistem testi, kullanıcı testi.
- **Test adımları ile geliştirme adımları arasındaki ilişki:** Birim testi için o birimin kodlaması tamamlanmış olmalıdır. Bütünlük testini yapabilmek için tasarım belgesinin elde olması gerekir. Sistem testinde gereksinim belgesine, yani gereksinim ve ürün spesifikasyonuna ihtiyaç vardır. Kullanıcı testinde gereksinim analizindeki fonksiyonel gereksinimlere ihtiyaç vardır.
- **Doğrulama ve geçerlemenin nerede yapıldığı:** Örneğin bütünlük testinde doğrulama yapılırken tasarım spesifikasyonunun nasıl kullanıldığı.

Test adımı	Ağırlıklı yapılan
Birim testi	Doğrulama (verification); test tipi saydam kutu
Bütünlük testi	Doğrulama (verification)
Sistem testi	Doğrulama ve geçerleme
İşlemsel kullanım (kabul)	Yalnızca geçerleme (validation)

Birim testinde birim, en küçük kod parçasıdır: bir modül, bir alt program ya da bir prosedür olabilir. Birimin veri yapıları doğru çalışıyor mu, algoritma akışı nasıl, döngüler doğru kurulmuş mu, bunlar incelenir. Bu doğrulama sırasında test tipi olarak saydam kutu testi yapılır.

19.2 Doğrulama ve Geçerleme

Hoca bu tanımları derste bir soru olarak da sordu ve cevabını kendisi verdi:

- **Geçerleme (validation):** “Doğru ürünü oluşturduk mu?” Örneğin kullanıcı dışarıdan bakar ve istediği ürünün oluşturulup oluşturulmadığını görür. Sistem testinde ürünün istenen özelliklere sahip olup olmadığına bakmak da geçerlemedir.
- **Doğrulama (verification):** “Ürünü doğru yolla oluşturduk mu?” Ürünü oluştururken algoritma hatası var mı, veri yapıları doğru kullanıldı mı, doğru yol izlendi mi, buna bakılır. Bunun için iç yapıya ihtiyaç vardır. Birim testinde ve bütünlük testinde ağırlıklı olarak doğrulama yapılır.

19.3 Tekrarın Geri Kalanı

Hoca tekrarı şöyle sürdürdü:

- Birim testi kapsamında test senaryosu, test sürücü ve kukla modül kavramları görüldü.
- Birim testinden sonra ara birim testi yapılır; modüller arasındaki ilişkileri ortaya çıkarmak için test veri grubu verilir.

- Bütünlük testinde modüller ara birimlerle birbirine bağlıdır. Birleştirmede veri kaybı, modüllerin birbirini ters etkilemesi, alt fonksiyonların beklentilerinin farklı olması ve hata toleranslarının büyümesi gibi sorunlar çıkabilir. Bütün olarak sınamak daha güç olduğundan artırmalı sınama, yukarıdan aşağı ya da aşağıdan yukarı yapılır.
- Regresyon testinde, sınanmış bir program parçasında değişiklik yapılırsa programın tümü yeniden test edilir.
- Sistem testinde düzeltme, güvenlik, dayanıklılık ve yetenek testleri yapılır.
- Onaylama testinde yazılımın doğru çalışıp çalışmadığı denenir, konfigürasyon elemanları incelenir ve en sonda kabul işlemi gerçekleşir.
- Saydam kutu testi iç yapının değerlendirilmesidir. Genellikle birim testinde, kısmen de bütünlük testinde uygulanır. Temel yol testinde yazılım akışındaki her temel birimin doğru akışta olup olmadığına ve akış yollarının sayısına bakılır; her akış yolu test edilir. Döngü testinde bütün döngüler artışlar, indisler, başlangıç ve bitiş noktaları açısından incelenir.
- Kara kutu testi, dışarıdan bakılan ve içerinin ayrıntılarıyla ilgilenilmeyen dış testtir. Uygun ve uygun olmayan girdiler verildiğinde sistemin nasıl cevap verdiğine bakılır. Eş değerli bölümlemede veri uzayından gruplar oluşturulup her gruptan veri gönderilir. Sınır değer analizinde test verisi daha çok girdi alanının kenarlarından seçilir.
- Son olarak keşif testi ve hata giderme (debug) görüldü.

Örnek Sorular

Bu derste kare yoktur. Aşağıdaki sorular hocanın derste sözlü olarak sorduğu sorulardır.

Soru 1 (derste sözlü, 00:42:15): Regresyon testi nedir?

> **Çözüm:** Bir öğrencinin cevabı: Bir uygulamanın hataları düzeltildikten sonra verilen yeni sürümünde, o hataların gerçekten düzeltilip düzeltilmediğinin kontrol edilmesidir. Hoca ekledi: Geliştirme sürecinde alt birimlerde bir hata olup değişiklik yapılırsa bütünlük testi tekrarlanır; bu da regresyon testidir. Genel tanım: Test edilmiş bir programda değişiklik veya ekleme yapıldığında programın tümünün yeniden test edilmesidir.

Soru 2 (derste sözlü, 01:31:02): Bir yazılım hazırladınız, kodladınız ve test edeceksiniz. Klasik yolla bakıldığında hangi sırada hangi testleri uygularsınız? (Test tiplerini değil, test adımlarını soruyorum.)

> **Çözüm:** Test süreci adımları sırasıyla birim testi, bütünlük testi, sistem testi ve kullanıcı testidir. Derste iki yanlış anlama düzeltildi. Bir öğrenci test yönetimi adımlarını saydı (planlama, test tasarımı ve senaryolar, testin yürütülmesi, hata raporlama). Hoca bunun "testi nasıl yönetirim?" sorusunun cevabı olduğunu söyledi: Önce planlanır, test verisi ve test senaryoları oluşturulur, senaryolar uygulanır, sonra test raporu ve test sonuç raporu yazılır. Aynı öğrenci adımları sayarken kara kutu testini de bir adım olarak saydı; hoca kara kutu testinin bir adım değil, bir test tipi olduğunu belirtti. Ayrıca birim testinden hemen sonra regresyon testi yapılmaz.

Soru 3 (derste sözlü, 01:38:51): Saydam kutu test tipinde hangi testler vardır? Temel yol testinde neye bakıyoruz ve yolları bulup ne yapıyoruz?

> **Çözüm:** Bir öğrencinin cevabı: Saydam kutu testleri temel yol testi ve döngü testidir. Temel yol testinde mantıksal karmaşıklık ölçülür, döngü testinde döngüler kontrol edilir. Sorunun ikinci kısmı derste öğrencilerden cevap alınamadan araya girildi.
>
> **Çözüm (derste verilmedi):** Dersin anlatımına göre temel yol testinde programdaki akış yollarının sayısı McCabe döngüsel karmaşıklık değeriyle bulunur. Her temel yol için o yolu geçecek test verisi hazırlanır ve program temel yol sayısı kadar çalıştırılır. Böylece her bağımsız yol en az bir kez test edilmiş olur.

Soru 4 (derste sözlü, 01:39:31): Kara kutu test tipleri nelerdir?

> **Çözüm (derste verilmedi):** Bu soruya derste öğrenciden cevap gelmedi. Dersin anlatımına göre başlıca kara kutu test yöntemleri eş değerli bölümleme, sınır değer analizi, neden-sonuç grafiği çizimi ve veri onaylama testidir. Keşif testi de dinamik bir kara kutu testi olarak anlatıldı.

Soru 5 (derste sözlü, 02:06:32): Çevik (agile) yöntemlerde gereksinim analizinden gelen hatalar neden azalmış olabilir?

> **Çözüm:** Öğrencilerden gelen cevaplar: gereksinim ile kod arasındaki sürenin kısa olması, gereksinim analizinin kısa olması, iteratif ve küçük adımlarla ilerlenmesi (özellikle XP). Hocanın aradığı asıl cevap: Müşteri ya da müşteri temsilcisi ekibin aktif bir parçasıdır. Müşteri ürünü yalnızca sonda değil, aşama aşama her halinde görür. Bu da müşterinin isteğiyle ilgili yanlış anlaşılmaları azaltır.

Soru 6 (derste sözlü, 02:22:30): Geçerleme ve doğrulama (validation ve verification) nedir?

> **Çözüm:** Öğrenciden cevap gelmeyince hoca kendisi cevapladı. Geçerleme “doğru ürünü oluşturduk mu?” sorusudur; kullanıcı ürüne dışarıdan bakar, sistem testinde ürünün istenen özelliklere sahip olup olmadığı denetlenir. Doğrulama “ürünü doğru yolla oluşturduk mu?” sorusudur; algoritma, veri yapıları ve izlenen yol denetlenir ve bunun için iç yapıya ihtiyaç vardır. Birim ve bütünlük testinde ağırlıklı olarak doğrulama, işlemsel kullanımda yalnızca geçerleme, sistem testinde ikisi birden yapılır.

Dersin Toparlaması

Bu derste geçen hafta başlanan test süreci adımları tamamlandı ve test tipleri görüldü. Test süreci adımları testin sırasını verir: Kodu yazanın yaptığı birim testi, modülleri ara birimleri üzerinden birleştiren bütünlük testi ve onu tamamlayan regresyon testi, yazılımı donanım, işletim sistemi ve veritabanıyla birlikte deneyen sistem testi, en sonda da onaylama testi, alfa ve beta testleri ve konfigürasyon incelemesiyle tamamlanan kabul muayenesi. V model bu adımların sırasını, her adımın hangi geliştirme belgesine dayandığını ve doğrulama ile geçerlemenin nerede yapıldığını birlikte gösterir.

Test tipleri ise her adımda hangi bakış açısıyla test yapılacağını anlatır. Saydam kutu testi iç yapıya bakar; temel yol testiyle bağımsız akış yollarının her biri, döngü testiyle de bütün döngüler denenir. Kara kutu testi iç yapıyla ilgilenmez; girdi uzayını eş değerli sınıflara bölerek ve sınır değerlerini seçerek, anormal girdileri de bilerek göndererek sistemin dışarıya verdiği tepkileri inceler. İki tip birbirinin alternatifi değil tamamlayıcısıdır. Ders, test ile hata giderme arasındaki farkla ve hata düzeylerine göre verilen teslim kararlarıyla kapandı. Temel yol testinin temelindeki McCabe döngüsel karmaşıklık hesabı bir sonraki hafta yazılım ölçümü konusunda, ardından test yönetimi işlenecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.